Weekly Work Report – DevOps Intern

Week 4

Name: Kailas R Pillai

Day 1 - 05/01/2026

Tasks Completed Today:

- **Task1:** Familiarize Terraform basics, Learn how terraform works, fundamental concepts and lifecycle.
- **Task2:** Familiarize aws configure --profile

Learnings:

- **Terraform**
  - Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp.
  - Allows to define, provision, and manage infrastructure using declarative configuration files.
- **Terraform Working**
  - Terraform infrastructure code is written using **.tf** files
  - Terraform Core reads the configuration
  - Providers interact with cloud APIs
  - Infrastructure is created or updated
- **Terraform Concepts**
  - **Provider**
    - Providers allow Terraform to interact with cloud platforms.
    - provider "aws" {
      region = "ap-south-1"
      }
  - **Resource**
    - Resources represent infrastructure components.
    - resource "aws_instance" "example" {
      ami = "ami-xxxx"
      instance_type = "t2.micro"
      }
  - **Variables**
    - Variables are used to define values in configuration.

        - variable "instance_type" {
          default = "t2.micro"
          }

    - **Outputs**
        - Outputs display useful information after resource creation.
        - output "public_ip" {
          value = aws_instance.example.public_ip
          }

    - **State File**
        - Terraform maintains a state file (terraform.tfstate) to track infrastructure details. It helps Terraform understand what already exists and what needs to change.

- **Terraform Lifecycle**
    - Terraform operates through the following lifecycle stages:
    - **terraform init**
        - Initializes the project and downloads providers.
    - **terraform plan**
        - Shows execution plan without making changes.
    - **terraform apply**
        - Creates or updates infrastructure.
    - **terraform destroy**
        - Removes all managed infrastructure.
- **AWS Configure**
    - aws configure --profile is used to set up AWS credentials and settings for a specific named profile instead of the default one.
    - aws configure --profile profile_name
    - IAM user is required for configuring.
    - Secret key and Secret access key is generated for IAM user. Keys are used for configuring.
    - **aws s3 ls --profile dev:** It lists all S3 buckets in the AWS account using the dev profile credentials.
    - AWS CLI must be installed for aws configure
    - **aws configure list-profiles:** It lists all AWS CLI profiles configured on your system.
    - **aws configure list --profile dev:** Shows the AWS credentials and region currently active for the profile.

Day 2- 06/01/2026

Tasks Completed Today:

- **Task1:** Created terraform modules for creating VPC, S3, EC2, RDS, cloudfront and ECS.
- **Task2:** Learned production environment setup of terraform.
- **Task3:** Familiarized terraform good practices and setup terraform in a production environment.

Learnings:

- **Terraform modules**
  - Created a terraform folder with Modules for VPC, S3, EC2, Cloudfront, ECS and RDS.
  - Architecture for each resource is stored inside a subfolder named module.
  - Created main.tf in the root folder to specify the source to each module.
  - Created output.tf output the results of output.tf in each module.

```
module.rds.aws_db_instance.this: Still creating... [04m10s elapsed]
module.rds.aws_db_instance.this: Still creating... [04m20s elapsed]
module.rds.aws_db_instance.this: Still creating... [04m30s elapsed]
module.rds.aws_db_instance.this: Still creating... [04m40s elapsed]
module.rds.aws_db_instance.this: Still creating... [04m50s elapsed]
module.rds.aws_db_instance.this: Creation complete after 4m56s [id=db-XSDM2OMTIK5CGHETVRDI4R26NQ]

Apply complete! Resources: 18 added, 0 changed, 0 destroyed.

Outputs:

cloudfront_domain = "d2jr1k8awcixsb.cloudfront.net"
ec2_instance_id = "public-ec2"
rds_username = "admin"
s3_bucket_name = "kailas-demo-bucket"
vpc_id = "public-vpc"
```

- **Terraform Production Environment Setup**
  - Configured AWS authentication using AWS CLI profiles by generating Access Key and Secret Access Key for that user.
  - Created a DynamoDB table earlier to enable Terraform state locking.
  - A state lock is a mechanism that ensures only one Terraform operation can modify the state at a time.
  - Prevents multiple Terraform runs from modifying state simultaneously.
  - Avoids state corruption in team environments.
  - Terraform tries to write a lock item into DynamoDB
  - If successful, lock is acquired
  - If lock exists, Terraform stops and waits.
  - After apply, lock is deleted
  - Created an IAM Role for EC2 with Trust policy allowing ec2.amazonaws.com
  - Attached managed policy: AmazonSSMManagedInstanceCore

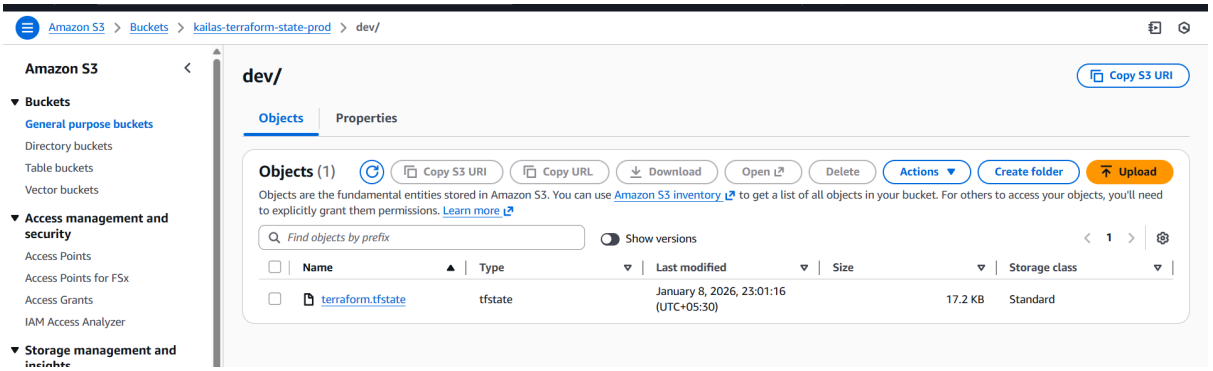
- Created an Instance Profile and attached it to EC2.
- EC2 instances borrow permissions from IAM roles.
- No access keys are stored on the instance.
- STS provides temporary credentials automatically. Instances are connected through Session Manager in the production environment.
- S3 is used to store Terraform state remotely and centrally.
- Created backend.tf which defines where and how Terraform stores its state

C: > Users > kaila > Downloads > terraform > backend.tf > terraform

```
terraform {
  backend "s3" {
    bucket         = "kailas-terraform-state-prod"
    key            = "dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

- Resource creation was successful after terraform apply stage and the instance was connected using session manager.

- Terraform.tfsate was stored in S3 bucket specified in the backend.



Day 3- 07/01/2026

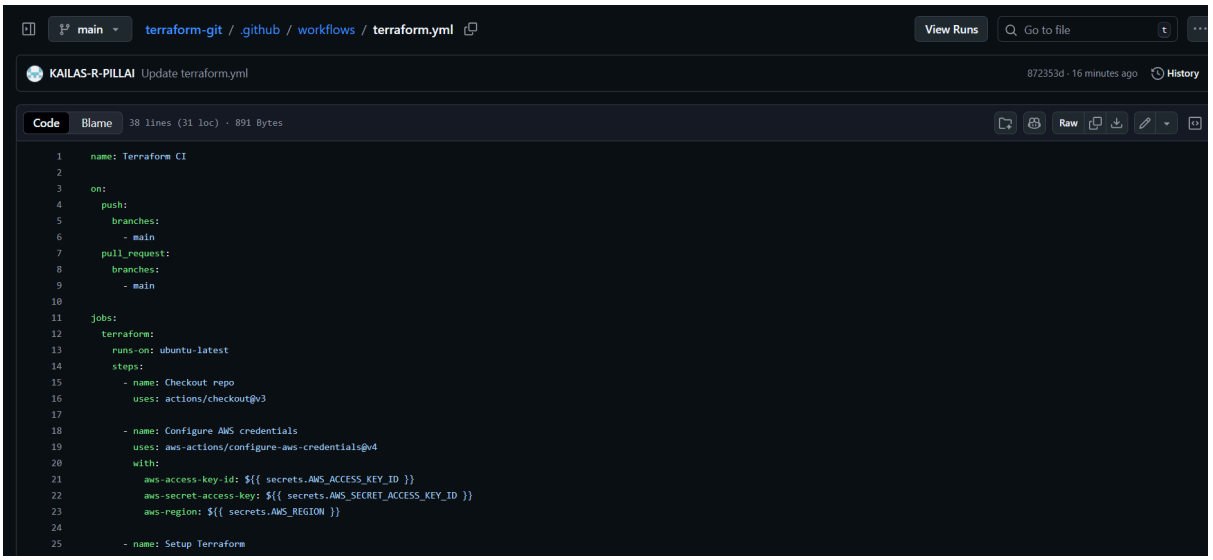
Tasks Completed Today:

- **Task1:** Learned github actions for IAC. Connected github actions and AWS with IAM roles.
- **Task2:** Created EC2 instance with Terraform.Destroying Infrastructure with Terraform
  - Understanding Terraform State files

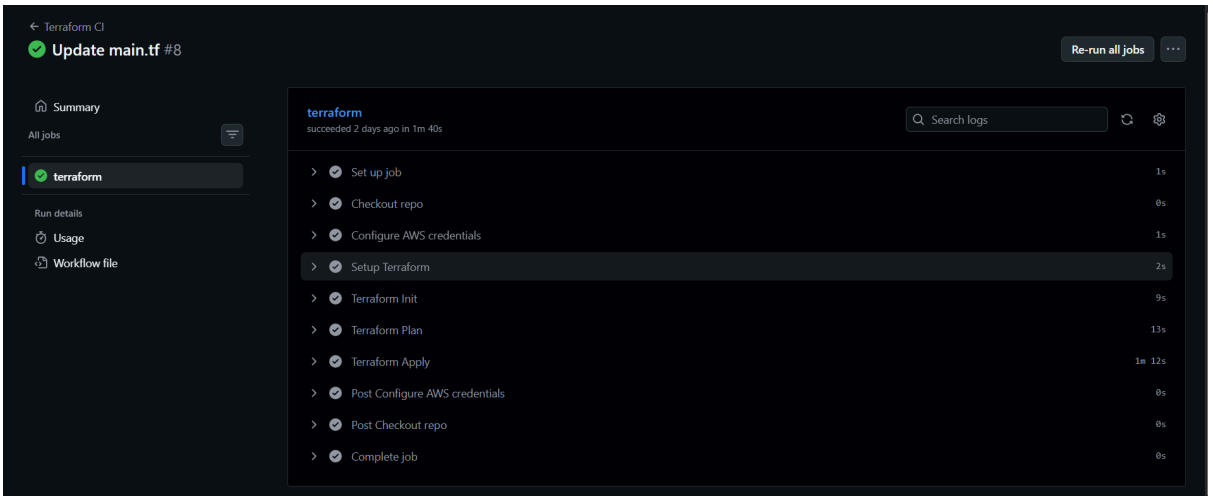
- Destroyed Infrastructure with Terraform
- Understanding Terraform State files
- Terraform Provider Versioning
- Types of Terraform Providers

Learnings:

- **Github Actions for IAC**
  - Pushed a terraform folder containing modules for EC2 and VPC into Github.
  - Created YAML CI/CD inside .github/workflows for CI/CD using github actions.
  - Configured AWS by providing Access key and secret key inside secrets and variables.



  - Github actions were successfully run and the resources were successfully created.



- **Creating first EC2 instance with Terraform**
  - Created Infrastructure for EC2 inside a terraform folder.
  - Created ec2.tf defining the ec2-ssm-role for connecting and the security groups. Specified the name of the instance inside the tag block.
  - Specified the providers and region in provider.tf

C:\Users\kaila\Downloads\terraform_ec2\providers.tf

```
terraform {
  required_version = ">= 1.3.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region  = "ap-south-1"
  profile = "kailas"
}
```

  - Initialized the folder using terraform init.
  - Previewed what Terraform is going to change before it actually changes anything using terraform plan.

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_instance_profile.ssm_profile will be created
  + resource "aws_iam_instance_profile" "ssm_profile" {
      + arn         = (known after apply)
      + create_date = (known after apply)
      + id          = (known after apply)
      + name        = "ec2-ssm-instance-profile"
      + name_prefix = (known after apply)
      + path        = "/"
      + role        = "ec2-ssm-role"
      + tags_all    = (known after apply)
      + unique_id   = (known after apply)
    }
```

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform apply

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_role.ssm_role: Creating...
aws_iam_role.ssm_role: Creation complete after 2s [id=ec2-ssm-role]
aws_iam_role_policy_attachment.ssm_policy: Creating...
aws_iam_instance_profile.ssm_profile: Creating...
aws_iam_role_policy_attachment.ssm_policy: Creation complete after 1s [id=ec2-ssm-role-20260110075534844500000000
1]
aws_iam_instance_profile.ssm_profile: Creation complete after 9s [id=ec2-ssm-profile]
aws_instance.my_ec2: Creating...
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 14s [id=i-0597a215c63d73cc1]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

- After terraform apply the instance was created in AWS.

MyFirstEC2	i-0dd661d475d42a04c	Running	t2.micro	2/2 checks passed	View alarms +	ap-south-1b	ec2-13-233-

- After creating destroyed all the infrastructure using terraform destroy.

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform destroy
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_iam_role_policy_attachment.ssm_policy: Destroying... [id=ec2-ssm-role-20260110071909711800000001]
aws_instance.my_ec2: Destroying... [id=i-0dd661d475d42a04c]
aws_iam_role_policy_attachment.ssm_policy: Destruction complete after 1s
aws_instance.my_ec2: Still destroying... [id=i-0dd661d475d42a04c, 00m10s elapsed]
aws_instance.my_ec2: Still destroying... [id=i-0dd661d475d42a04c, 00m20s elapsed]
aws_instance.my_ec2: Still destroying... [id=i-0dd661d475d42a04c, 00m30s elapsed]
aws_instance.my_ec2: Destruction complete after 31s
aws_iam_instance_profile.ssm_profile: Destroying... [id=ec2-ssm-profile]
aws_iam_instance_profile.ssm_profile: Destruction complete after 0s
aws_iam_role.ssm_role: Destroying... [id=ec2-ssm-role]
aws_iam_role.ssm_role: Destruction complete after 2s

Destroy complete! Resources: 4 destroyed.
PS C:\Users\kaila\Downloads\terraform_ec2>
```

Day 4- 08/01/2026

Tasks Completed Today:

- **Task1:** Learned about Attributes and Output Values in Terraform.
- **Task2:** Familiarized about Provisioners in Terraform.
- **Task3:** Familiarized DRY principle.

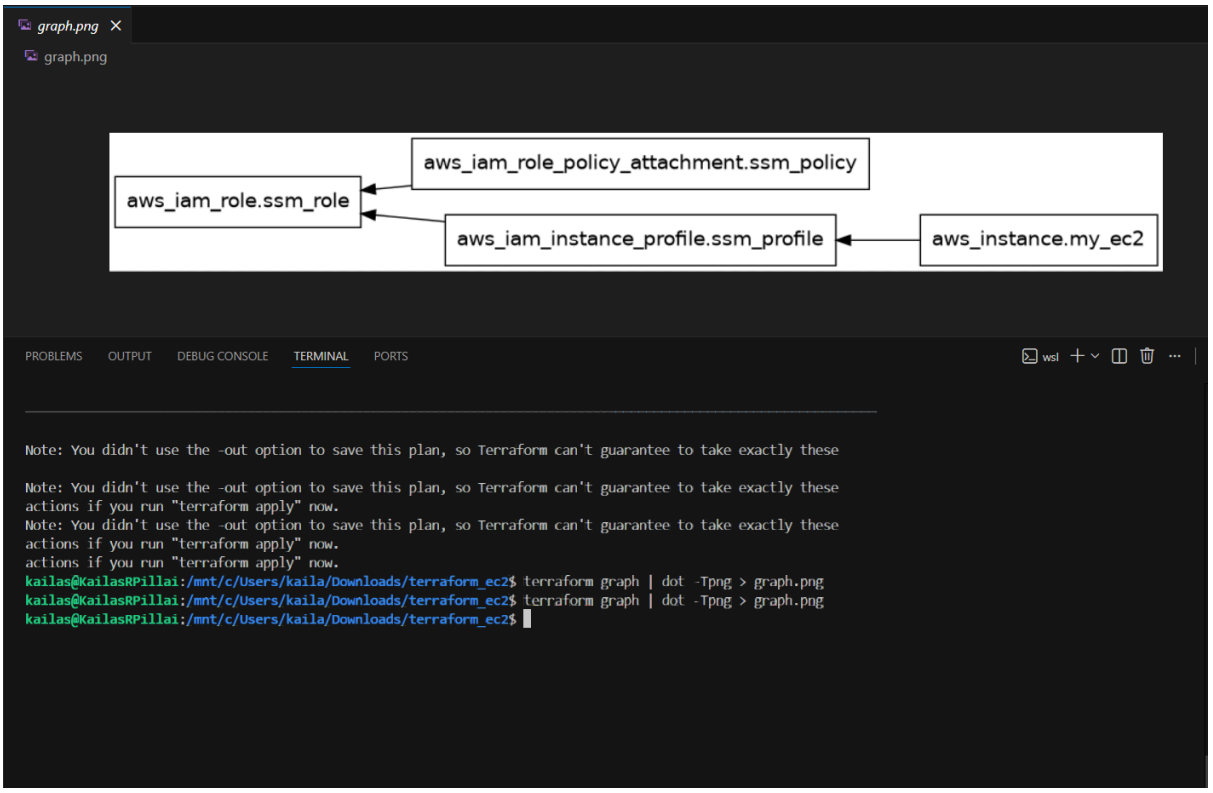
Learnings:

- **Attributes and Output Values in Terraform**

- **Resource Attributes:** Every Terraform resource exposes attributes after creation.
- resource "aws_instance" "web" {
  ami = "ami-xxxxxxxxxx"
  instance_type = "t2.micro"
  }

- Common attributes after creation can be:
  - aws_instance.web.id
  - aws_instance.web.public_ip
- **Output Values:** Outputs are used to display the results after terraform apply.
- output "instance_id" {
  value = aws_instance.web.id
  }
- This displays the instance ID as output after terraform apply.
- **Terraform Variables:** Variables make Terraform reusable
- variable "instance_type" {
  type = string
  default = "t2.micro"
  }
- We call this variable by **instance_type = var.instance_type**
- Variables can be defined by default in the terraform code.
- This has least priority if the variables are set via CLI.
- Highest priority is by defining them inside terraform.tfvars which is easy to reuse as it has all the variables defined and for another configuration only the value needs to be changed.
- **Data Types for Variables**
  - **String :** “t2.micro”
  - **Number :** 10
  - Bool : true/false
  - List : [“a”,”b”]
  - Map : {env = “dev”}
- **Count Index:** Used to create multiple identical resources.
  - resource "aws_instance" "web" {
    count = 3
    ami = "ami-XXXXXXX"
    }
  - To show the instance id of the particular indec
  - output "instance_ids" {
    value = aws_instance.web[count.index].id
    }
  - aws_instance.web[0].id will show the instance ID of the instance in the zeroth index.

- **Conditional Expressions**
  - instance_type = var.env == "prod" ? "t3.large" : "t2.micro"
  - That is if the environment is prod, then use instance type t3.large, else use t2.micro.
- **Tainting Resources:** Forces to recreate.
  - terraform taint aws_instance.web: to recreate
  - terraform untaint aws_instance.web: to undo recreation
- **Terraform Graph**
  - Terraform graph is generated using the command terraform graph.
  - This outputs DOT format text.
  - Converted DOT text can be used to generate PNG.



- **Saving Terraform Plan to File**
  - To save a Terraform plan to a file, you use the -out option with terraform plan.

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform plan -out=tfplan
      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)
    }

Plan: 4 to add, 0 to change, 0 to destroy.

------------------------------------------------------------------------

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
    terraform apply "tfplan"
```

- During terraform apply the saved tfplan can be used to create the architecture.

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform apply tfplan
aws_iam_role.ssm_role: Creating...
aws_iam_role.ssm_role: Creation complete after 3s [id=ec2-ssm-role]
aws_iam_role_policy_attachment.ssm_policy: Creating...
aws_iam_instance_profile.ssm_profile: Creating...
aws_iam_role_policy_attachment.ssm_policy: Creation complete after 1s [id=ec2-ssm-role-20260110182856219800000001]
aws_iam_instance_profile.ssm_profile: Creation complete after 9s [id=ec2-ssm-profile]
aws_instance.my_ec2: Creating...
aws_instance.my_ec2: Still creating... [00m10s elapsed]
aws_instance.my_ec2: Creation complete after 15s [id=i-007593059b5186e05]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

- **Terraform Provisioners**
  - Provisioners are used in Terraform to execute scripts or commands on a local machine or a remote resource after it is created.
  - **Types of Provisioners in Terraform**
    - **local-exec:** Runs commands on the machine where Terraform is executed
    - **Remote-exec:** Runs commands on the remote resource (e.g., EC2 instance). Requires SSH connection
  - Used local-exec to install some packages to an instance after it is created using terraform apply.
  - Checked if the package was successfully installed by verifying its version.

```
Session ID: kailas-8xgqt7qvz58kx5g5ooj9fq8acu   Shortcuts   Instance ID: i-08730084b6684b8a4   Terminate

sh-5.2$ nginx -v
nginx version: nginx/1.28.0
sh-5.2$
```

- **DRY Principle**
    - The DRY (Don't Repeat Yourself) principle means avoiding duplicating code. Instead of writing the same logic multiple times, you write it once and reuse it.
    - With DRY:
        - Reusable modules
        - Easier maintenance
        - Clean and scalable infrastructure
    - Instead of writing the same code we can just write it once and define that as a module.
        - module "dev_ec2" {
          source = "./ec2"
          instance_type = "t2.micro"
          }

          module "prod_ec2" {
          source = "./ec2"
          instance_type = "t2.medium"
          }
- Defined EC2 infra using terraform and specified the source to EC2 infra inside the module block.

main.tf

main.tf > module "rds" > password

```
module "vpc" {
source = "./modules/vpc"
vpc_cidr = "10.0.0.0/16"
public_subnet_cidr = "10.0.1.0/24"
az1 = "ap-south-1a"
az2 = "ap-south-1b"
}


module "ec2" {
source = "./modules/ec2"
subnet_id = module.vpc.public_subnet_id
ami_id = "ami-04808bdb94be6720e"
}


module "s3" {
```

- **Terraform Registry**
    - The **Terraform Registry** is a public repository of Providers (AWS, Azure, GCP) and Modules (VPC, EC2, EKS)
    - Instead of writing the whole code we can use terraform registry which has production ready modules.
    - Example:

```
module "vpc" {
source = "terraform-aws-modules/vpc/aws"
version = "5.1.0"
name = "my-vpc"
cidr = "10.0.0.0/16"
}
```

Day 5 - 09/01/2026

Tasks Completed Today:

- **Task1:** Familiarized SLA, SLI and SLO.
- **Task2:** Integrated Terraform with Git for team management.
- **Task3:** Learned to handle Access Key & Secret Key in the right way in Providers.

Learnings:

- **SLI:** An SLI is a quantitative metric that shows how well a service is performing.
    - Instance uptime, request latency, error rate
- **SLO:** An SLO defines the acceptable level for an SLI over a specific time period. (99.9% availability per month).
- **SLA:** An SLA is a formal contract between the service provider and the customer. It refers to AWS guarantees.
    - It defines:
        - Guaranteed service levels
        - Measurement method
        - Penalties if targets are not met

- **Integrating Terraform with GIT for Team Management**
    - Created a terraform folder containing modules for VPC and EC2.
    - Created backend.tf to store remote state in S3
    - State locking is done using DynamoDB
    - Integrated DynamoDB with S3 for state locking.
    - Pushed code into Github.

main terraform-git / backend.tf

KAILAS-R-PILLAI Added Terraform files

Code Blame 9 lines (9 loc) · 229 Bytes

```
terraform {
  backend "s3" {
    bucket         = "kailas-terraform-state-prod"
    key            = "dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

- Did not commit .tfstate as it contains sensitive data. Added .terrafrom and .tfstate and .tfstate.backup in .gitignore.
- We import shared or pre-existing resources into Terraform state and then create new compute resources using Terraform.
- Imported a pre-existing ssm-role using terraform import.

```
PS C:\Users\kaila\Downloads\terraform_ec2> terraform apply
  Error: creating IAM Role (ec2-ssm-role): operation error IAM: CreateRole, https response error StatusCode: 409, Req
estID: 57764cc5-8489-45b0-911f-960f32a3256a, EntityAlreadyExists: Role with name ec2-ssm-role already exists.

    with aws_iam_role.ssm_role,
    on ec2.tf line 2, in resource "aws_iam_role" "ssm_role":
     2: resource "aws_iam_role" "ssm_role" {


PS C:\Users\kaila\Downloads\terraform_ec2> terraform import aws_iam_role.ssm_role ec2-ssm-role
aws_iam_role.ssm_role: Importing from ID "ec2-ssm-role"...
aws_iam_role.ssm_role: Import prepared!
  Prepared aws_iam_role for import
aws_iam_role.ssm_role: Refreshing state... [id=ec2-ssm-role]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.
```

- **Handling Access Keys and Secret Access Keys the right way in Providers**
  - Should not define Access Key and Secret Access key in .tf files because it leads to security risks when committed to git to even exposed publicly.
  - Best practice is to Use AWS CLI Profiles (aws configure --profile <profile_name>) and define the profile in .tf files.
  - Another way is to define them as secrets or environment variables and export them.
- **Resources in Multiple Regions**
  - Different resources can be defined in different regions.

- For instance, EC2 in ap-south-1 and S3 in us-east-1.
- provider "aws" {
  region = "ap-south-1"
  }
  provider "aws" {
  alias = "us"
  region = "us-east-1"
  }

- Alias is used for an additional, named instance of the same provider with a different configuration.
- Aws is the default provider aws.us is the aliased provider.

- **Multiple AWS Accounts**
  - provider "aws" {
    profile = "dev"
    region = "ap-south-1"
    }
    provider "aws" {
    alias   = "prod"
    profile = "prod"
    region = "ap-south-1"
    }

  - resource "aws_instance" "prod_ec2" {
    provider = aws.prod
    }
  - Here two accounts are defined. One named **prod** and one named **dev**.

- **Assume roles**
  - STS is an AWS service that gives temporary credentials.
  - AssumeRole is the act of borrowing permissions from a role attached to it for a short time.

```
resource "aws_iam_role" "ssm" {
name = "ec2-ssm-profile"


assume_role_policy = jsonencode({
Version = "2012-10-17"
Statement = [{
Effect = "Allow"
Principal = { Service = "ec2.amazonaws.com" }
Action = "sts:AssumeRole"
}]
})
}


resource "aws_iam_role_policy_attachment" "ssm" {
role = aws_iam_role.ssm.name
policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}
```

  - When an EC2 instance is launched with an IAM role, AWS automatically allows the EC2 service to call STS AssumeRole based on the role's trust policy.
  - STS then issues temporary credentials which the instance uses to access AWS services like Systems Manager, eliminating the need for stored access keys or SSH.
  - Prevents the issue of keys getting leaked. Keys are temporary and rotation is automated.
  - Access to the instance is possible as long as the role exists.
- **Sensitive parameter**
  - A sensitive parameter is a value that Terraform intentionally hides from output and logs to prevent accidental exposure.
  - variable "db_password" {
    type = string
    sensitive = true
    }
  - Now only terraform knows the value and it will not display the value.